Lecture Notes: TwinCAT 3

Introduction to PLC Programming with TwinCAT 3

March 17, 2026

Contents

1 Overview and Objectives **2**

1.1 Core Philosophy 2

2 The TwinCAT 3 Ecosystem **2**

2.1 Basic Group (Parts 2–12) 2

2.2 Advanced Group (Parts 13–17) 4

3 Programming Logic and Instructions **5**

4 Hardware Interaction (I/O) **8**

5 Scope and Limitations **8**

Quick Reference Card **11**

1 Overview and Objectives

This tutorial series provides a comprehensive introduction to **TwinCAT 3 software development**, bridging the gap between traditional electrical engineering and modern software engineering practices. It is designed for both beginners and those with a software background who are new to industrial automation.

1.1 Core Philosophy

- **Software Engineering vs. Electrical Engineering:** Traditionally, PLC tools were rooted in electrical engineering (replacing physical relays). TwinCAT 3 represents a shift toward "real" software engineering.
- **Focus on Structured Text (ST):** While the IEC 61131-3 standard defines five languages (SFC, LD, FBD, IL, and ST), this series focuses exclusively on **Structured Text** because it is the most similar to high-level programming languages like C and C++.

Language	Abbreviation	Type	Origin
Structured Text	ST	Textual	Pascal/C-like
Ladder Diagram	LD	Graphical	Relay logic
Function Block Diagram	FBD	Graphical	Signal flow
Instruction List	IL	Textual	Assembly-like
Sequential Function Chart	SFC	Graphical	State machines

Table 1: IEC 61131-3 Programming Languages

> **Note:** Structured Text (ST) is recommended for programmers with a software background because it supports familiar constructs like loops, conditionals, and object-oriented programming concepts that are common in languages like C, C++, and Python.

2 The TwinCAT 3 Ecosystem

The tutorial is divided into two main sections: **Basic Core Functionality** and **Advanced Ecosystem Understanding**.

2.1 Basic Group (Parts 2–12)

- **Environment (XAE):** Setting up the development environment and defining real-time tasks.
- **Real-Time Execution:** Investigating how a PLC defines a task and executes a program.
- **Data Handling:** Exploring standard data types and **Structures** for grouping logical variables.
- **Code Organization:**
  - **Functions:** Getting data in and out of modular logic.
  - **Function Blocks:** These are the building blocks for **Object-Oriented Programming (OOP)** in the PLC world, acting as the equivalent of classes.
- **Standard Libraries:** Utilizing the `Tc2_Standard` library for essential functions like timers and triggers.

Example: Basic Structured Text Variable Declaration

```
// PROGRAM: MAIN
// Demonstrates basic variable declaration in Structured Text
// Variables are declared within VAR...END_VAR blocks

PROGRAM MAIN
VAR
    // ====================================================
    // BOOLEAN TYPE
    // Used for TRUE/FALSE logic, digital I/O, flags
    // ====================================================
    bMotorRunning    : BOOL := FALSE;     // Motor status flag
    bStartButton     : BOOL;               // Start pushbutton input
    bStopButton      : BOOL;               // Stop pushbutton input
    bAlarmActive     : BOOL := FALSE;      // Alarm indicator

    // ====================================================
    // INTEGER TYPES
    // Various sizes for different numeric ranges
    // ====================================================
    nCounter         : INT := 0;           // 16-bit signed (-32768 to 32767)
    nProductCount    : UINT := 0;          // 16-bit unsigned (0 to 65535)
    nBatchNumber     : DINT := 0;          // 32-bit signed integer
    nTotalParts      : UDINT := 0;         // 32-bit unsigned integer
    nSmallValue      : SINT := 0;          // 8-bit signed (-128 to 127)
    nByteValue       : USINT := 0;         // 8-bit unsigned (0 to 255)

    // ====================================================
    // FLOATING POINT TYPES
    // For decimal/fractional values
    // ====================================================
    fTemperature     : REAL := 20.5;       // 32-bit float (single precision)
    fPressure        : REAL := 101.325;    // Atmospheric pressure in kPa
    fPreciseValue    : LREAL := 3.14159265358979;  // 64-bit double precision

    // ====================================================
    // STRING TYPE
    // Text data with specified maximum length
    // ====================================================
    sProductName     : STRING(50) := 'Widget A';    // 50 character max
    sStatusMessage   : STRING(255);                  // 255 character max
    sOperatorName    : STRING := 'Default';          // Default 80 chars

    // ====================================================
    // TIME TYPES
    // For delays, timers, and time-based logic
    // ====================================================
    tCycleTime       : TIME := T#100MS;    // 100 milliseconds
    tDelayDuration   : TIME := T#5S;       // 5 seconds
```

```
    tLongDelay       : TIME := T#1H30M;    // 1 hour 30 minutes

    // ==================================================
    // ARRAYS
    // Collections of same-type values
    // ==================================================
    aTemperatures    : ARRAY[0..9] OF REAL;          // 10 temperature readings
    aDigitalInputs   : ARRAY[1..8] OF BOOL;          // 8 digital inputs
    aBatchData       : ARRAY[0..99] OF INT;          // 100 integer values

END_VAR
```

```
// Additional declaration examples showing initialization

VAR
    // Initialized array
    aSensorLimits : ARRAY[0..2] OF REAL := [0.0, 50.0, 100.0];

    // Constants (cannot be changed at runtime)
    CONSTANT
        c_PI          : LREAL := 3.14159265358979;
        c_MAX_SPEED   : INT := 1500;
        c_TANK_SIZE   : REAL := 1000.0;    // Liters
    END_VAR

    // Bit/Byte types for hardware interfacing
    byStatusByte    : BYTE := 16#00;      // 8 bits, hex notation
    wControlWord    : WORD := 16#0000;    // 16 bits
    dwDataWord      : DWORD := 16#00000000;  // 32 bits

END_VAR
```

Tip: Always initialize variables with meaningful default values. Uninitialized variables in TwinCAT default to zero/FALSE, but explicit initialization makes code more readable and prevents unexpected behavior.

2.2 Advanced Group (Parts 13–17)

- **Professional Development Practices:** Implementing **Version Control** using Git and performing **Test-Driven Development (TDD)** using the `TcUnit` framework.
- **Connectivity (ADS):** Using the **Automation Device Specification** (Beckhoff's middleware) to communicate between the PLC and external applications in C# or Linux-based C++.
- **Automation Interface:** Programmatically automating the configuration and deployment process.

Topic	Part	Key Concepts
Environment Setup	2	XAE, XAR, Tasks
Data Types	3–4	INT, REAL, STRING, Structures
Functions	5	Inputs, Outputs, Return values
Function Blocks	6–7	OOP, Methods, Properties
Libraries	8	Tc2_Standard, Timers, Triggers
Program Flow	9	IF/ELSE, CASE, FOR, WHILE
I/O Mapping	10–11	Hardware linking
State Machines	12	ENUM-based control
Version Control	13	Git, .gitignore
Testing	14	TcUnit, TDD
ADS	15	C#, C++ connectivity
Automation	16–17	ITcSysManager

Table 2: Tutorial Series Overview

3 Programming Logic and Instructions

The tutorial covers standard program flow instructions necessary for creating useful PLC software.

- **Conditional Logic:** Using `IF/ELSE` and `CASE` switches to manage program branches.
- **Iteration:** Utilizing `FOR` and `WHILE` loops for repetitive tasks.

Example: Structured Text Control Flow

```
// PROGRAM: ControlFlowExamples
// Demonstrates IF/ELSE statements and CASE switches in Structured
    Text

PROGRAM MAIN
VAR
    // Input variables
    fTemperature     : REAL := 25.0;       // Current temperature
    nMachineState    : INT := 0;           // Current state (0-4)
    bEmergencyStop   : BOOL := FALSE;      // Emergency stop button
    bAutoMode        : BOOL := TRUE;       // Automatic mode enabled

    // Output variables
    bHeaterOn        : BOOL;               // Heater control
    bCoolerOn        : BOOL;               // Cooler control
    bAlarm           : BOOL;               // Alarm output
    sStatusText      : STRING(100);        // Status message
    nFanSpeed        : INT;                // Fan speed percentage

    // Setpoints
    fTempSetpoint    : REAL := 22.0;       // Target temperature
    fHysteresis      : REAL := 2.0;        // Deadband
END_VAR

// ====================================================
// IF...THEN...ELSE Example: Temperature Control
// ====================================================
```

```

// First, check for emergency conditions (highest priority)
IF bEmergencyStop THEN
    // Emergency stop - shut everything down
    bHeaterOn := FALSE;
    bCoolerOn := FALSE;
    bAlarm := TRUE;
    sStatusText := 'EMERGENCY STOP ACTIVE';

ELSIF NOT bAutoMode THEN
    // Manual mode - don't control automatically
    sStatusText := 'Manual Mode - No Auto Control';

ELSIF fTemperature > (fTempSetpoint + fHysteresis) THEN
    // Temperature too high - activate cooling
    bHeaterOn := FALSE;
    bCoolerOn := TRUE;
    bAlarm := FALSE;
    sStatusText := 'Cooling Active';

ELSIF fTemperature < (fTempSetpoint - fHysteresis) THEN
    // Temperature too low - activate heating
    bHeaterOn := TRUE;
    bCoolerOn := FALSE;
    bAlarm := FALSE;
    sStatusText := 'Heating Active';

ELSE
    // Temperature within acceptable range
    bHeaterOn := FALSE;
    bCoolerOn := FALSE;
    bAlarm := FALSE;
    sStatusText := 'Temperature OK';

END_IF


// ====================================================
// CASE Statement Example: State Machine Control
// ====================================================

CASE nMachineState OF

    0:  // IDLE State
        sStatusText := 'State 0: IDLE';
        nFanSpeed := 0;
        bHeaterOn := FALSE;

    1:  // STARTUP State
        sStatusText := 'State 1: STARTUP';
        nFanSpeed := 25;
        // Transition logic would go here

    2:  // RUNNING State
        sStatusText := 'State 2: RUNNING';
        nFanSpeed := 75;

    3:  // COOLDOWN State
```

```
        sStatusText := 'State 3: COOLDOWN';
        nFanSpeed := 100;
        bHeaterOn := FALSE;

    4:  // ERROR State
        sStatusText := 'State 4: ERROR';
        nFanSpeed := 0;
        bAlarm := TRUE;

ELSE
    // Unknown state - go to error
    sStatusText := 'UNKNOWN STATE!';
    nMachineState := 4;  // Force to error state

END_CASE
```

```
// Loop Examples: FOR and WHILE

VAR
    i               : INT;
    nSum            : INT := 0;
    aValues         : ARRAY[0..9] OF INT := [1,2,3,4,5,6,7,8,9,10];
    bSearching      : BOOL;
    nFoundIndex     : INT := -1;
    nSearchValue    : INT := 7;
END_VAR

// ====================================================
// FOR Loop: Sum all array elements
// ====================================================
nSum := 0;
FOR i := 0 TO 9 DO
    nSum := nSum + aValues[i];
END_FOR
// Result: nSum = 55

// ====================================================
// WHILE Loop: Search for a value
// ====================================================
i := 0;
bSearching := TRUE;
nFoundIndex := -1;

WHILE bSearching AND (i <= 9) DO
    IF aValues[i] = nSearchValue THEN
        nFoundIndex := i;
        bSearching := FALSE;  // Exit loop
    END_IF
    i := i + 1;
END_WHILE
// Result: nFoundIndex = 6 (0-based index where 7 is found)
```

Warning: WHILE loops can cause infinite loops if the exit condition is never met. Always ensure there is a guaranteed exit path, and consider adding a maximum iteration counter as a safety mechanism in production code.

4 Hardware Interaction (I/O)

To create a functional control system, the software must interact with the physical environment.

- **Actuation:** Controlling relays, mixers, or pneumatic systems.
- **Feedback:** Using sensors to provide inputs back to the PLC.
- **Mapping:** The process of connecting software variables to physical **Inputs and Outputs (I/O)**.

I/O Type	Direction	Example Device	Data Type
Digital Input	PLC ← Field	Pushbutton, Limit switch	BOOL
Digital Output	PLC → Field	Relay, Indicator lamp	BOOL
Analog Input	PLC ← Field	Temperature sensor	INT/REAL
Analog Output	PLC → Field	VFD speed reference	INT/REAL

Table 3: Common I/O Types in PLC Systems

Note: In TwinCAT, I/O mapping is typically done through the System Manager by linking PLC variables (marked with `AT %I*` for inputs or `AT %Q*` for outputs) to physical terminal addresses. Variables can also be linked graphically in the XAE environment.

5 Scope and Limitations

While TwinCAT is vast, some "huge" topics are acknowledged as being outside the primary scope of this introductory tutorial but essential for a complete understanding of the ecosystem:

- **Motion Control:** Controlling motors and axes.
- **HMI:** Creating front-end user interfaces.
- **Vision & Measurement:** Quality inspection via cameras and process analysis.

Example: Function Block Instantiation

```
// FUNCTION BLOCK Definition: FB_Motor
// Function Blocks are the OOP building blocks in PLC programming
// They encapsulate data (variables) and behavior (methods)
// Each instance maintains its own state - similar to objects in C++/
    Java

FUNCTION_BLOCK FB_Motor
VAR_INPUT
    bStart       : BOOL;      // Command to start motor
    bStop        : BOOL;      // Command to stop motor
    nSpeedRef    : INT;       // Speed reference (0-100%)
END_VAR

VAR_OUTPUT
    bRunning     : BOOL;      // Motor is running
```

```
    bFault       : BOOL;      // Motor fault detected
    nActualSpeed: INT;       // Actual speed feedback
END_VAR

VAR
    // Internal variables (private state)
    bEnabled     : BOOL;
    tRunTime     : TIME;      // Total run time
    nStartCount : UDINT;     // Number of starts

    // Internal timer instance
    tonRunTimer : TON;       // Timer to track run time
END_VAR

// Function Block Implementation (logic)
// This code executes each time the FB is called

// Start/Stop logic
IF bStop THEN
    bEnabled := FALSE;
ELSIF bStart AND NOT bFault THEN
    bEnabled := TRUE;
    nStartCount := nStartCount + 1;
END_IF

// Update outputs
bRunning := bEnabled AND NOT bFault;

// Simulate speed ramping (simplified)
IF bRunning THEN
    nActualSpeed := nSpeedRef;  // In reality, would ramp
ELSE
    nActualSpeed := 0;
END_IF

// Track run time
tonRunTimer(IN := bRunning, PT := T#24H);
```

```
// PROGRAM: MAIN
// Demonstrates declaring and calling Function Block instances
// Each instance is like an "object" with its own memory/state

PROGRAM MAIN
VAR
    // ==================================================
    // FUNCTION BLOCK INSTANCES
    // Each instance is independent - has its own variables
    // Similar to creating objects from a class
    // ==================================================

    // Motor controller instances
    fbMotor1     : FB_Motor;      // Conveyor motor
    fbMotor2     : FB_Motor;      // Pump motor
    fbMotor3     : FB_Motor;      // Fan motor

    // Standard library Function Block instances
    tonDelay     : TON;           // On-delay timer
    tofDelay     : TOF;           // Off-delay timer
```

```
    ctParts      : CTU;          // Up counter
    rtStartEdge  : R_TRIG;       // Rising edge detector
    ftStopEdge   : F_TRIG;       // Falling edge detector

    // Control variables
    bStartCmd    : BOOL;
    bStopCmd     : BOOL;
    nSpeedSetpoint : INT := 75;

    // Status variables
    bAnyMotorRunning : BOOL;
    nTotalStarts     : UDINT;
END_VAR

// ====================================================
// CALLING FUNCTION BLOCKS
// Must call each instance every scan to execute its logic
// Pass inputs, then read outputs after the call
// ====================================================

// Call Motor 1 - Conveyor
fbMotor1(
    bStart       := bStartCmd,
    bStop        := bStopCmd,
    nSpeedRef    := nSpeedSetpoint
);
// After call, outputs are available:
// fbMotor1.bRunning, fbMotor1.bFault, fbMotor1.nActualSpeed

// Call Motor 2 - Pump (different speed)
fbMotor2(
    bStart       := bStartCmd AND NOT fbMotor1.bFault,  // Interlock
    bStop        := bStopCmd,
    nSpeedRef    := 50
);

// Call Motor 3 - Fan (runs when either motor runs)
fbMotor3(
    bStart       := fbMotor1.bRunning OR fbMotor2.bRunning,
    bStop        := NOT (fbMotor1.bRunning OR fbMotor2.bRunning),
    nSpeedRef    := 100
);

// ====================================================
// USING STANDARD LIBRARY FUNCTION BLOCKS
// ====================================================

// Rising edge detection - detect button press
rtStartEdge(CLK := bStartCmd);
IF rtStartEdge.Q THEN
    // Start button was just pressed (rising edge)
END_IF

// On-delay timer - delay before action
tonDelay(IN := fbMotor1.bRunning, PT := T#5S);
IF tonDelay.Q THEN
    // Motor has been running for 5 seconds
END_IF
```

```

// Up counter - count parts
ctParts(CU := rtStartEdge.Q, RESET := bStopCmd, PV := 100);
// ctParts.CV = current count
// ctParts.Q = TRUE when CV >= PV

// ====================================================
// READING FUNCTION BLOCK OUTPUTS
// ====================================================
bAnyMotorRunning := fbMotor1.bRunning OR fbMotor2.bRunning OR
    fbMotor3.bRunning;
nTotalStarts := fbMotor1.nStartCount + fbMotor2.nStartCount +
    fbMotor3.nStartCount;
```

Tip: Function Blocks maintain their state between PLC scan cycles. This makes them ideal for implementing stateful logic like motors (running/stopped), timers (elapsed time), and counters (current count). Always call Function Block instances every scan cycle to ensure proper operation.

Quick Reference Card

Common Data Types

Type	Size	Range	Use Case
BOOL	1 bit	TRUE/FALSE	Digital I/O, flags
INT	16 bits	-32768 to 32767	Counters, small numbers
DINT	32 bits	±2.1 billion	Large integers
REAL	32 bits	±3.4e38	Temperatures, setpoints
STRING	Variable	Up to 255 chars	Messages, names
TIME	32 bits	T#0ms to T#49d	Timers, delays

Control Flow Syntax

```
// IF Statement
IF condition THEN
    // code
ELSIF other_condition THEN
    // code
ELSE
    // code
END_IF

// CASE Statement
CASE nVariable OF
    0: (* code for 0 *)
    1: (* code for 1 *)
    2..5: (* code for 2-5 *)
ELSE
    (* default *)
END_CASE
```

```

// FOR Loop
FOR i := 0 TO 10 BY 1 DO
    // code
END_FOR

// WHILE Loop
WHILE condition DO
    // code
END_WHILE
```

Common Standard Library FBs

Function Block	Purpose	Key Output
TON	On-delay timer	Q (done), ET (elapsed)
TOF	Off-delay timer	Q (done), ET (elapsed)
TP	Pulse timer	Q (pulse active)
CTU	Count up	CV (count), Q (done)
CTD	Count down	CV (count), Q (done)
R_TRIG	Rising edge	Q (edge detected)
F_TRIG	Falling edge	Q (edge detected)

Last Updated: March 17, 2026